HealthVisionPro App Guide

Getting started

HealthVisionPro helps review scan images, turn spoken symptoms into text, and view patient risk results. It also shows saved reports and charts. This guide covers the main screens and local setup. Reports and scores need professional review before they guide care.

Find your way around

Tab	What it does
Upload & Analyze	Upload a scan image and read an automatically written report.
Voice Assistant	Record speech, turn it into text and request a symptom summary.
History	Read image and voice reports saved in the database.
Graphical Insights	View charts from the bundled patient dataset.
Risk Predictor	Select an existing patient and run the saved risk model.

The sun or moon button switches the theme. The settings menu contains About, Contact Us, Help and Terms & Policies.

Analyze a scan image

1. Open Upload & Analyze and select Choose File.

2. Choose a medical scan image. PNG is a suitable starting format. A preview appears on the left.

3. Select Upload & Analyze and wait for processing to finish.

4. Read the report on the right. A completed report has four sections: Summary, What the Image Shows, Abnormalities and Impression.

You can select several images for a combined report. Each image is analyzed separately, and the text reports are joined together. This version does not draw finding boxes or return annotated scan images.

The screen accepts image files. The server also has a PDF processing route, but this screen has no PDF upload option.

A report from one image is saved to History when the database is available. Reports from multiple images are not saved to the database. A report can appear even if saving it failed, so check History if you need a saved copy.

Use voice and saved reports

Record and analyze symptoms

1. Open Voice Assistant and select Start Recording. Allow microphone access when your browser asks.

2. Speak clearly. Describe the symptoms, where they occur and how long they have lasted.

3. Select Stop Recording, then Transcribe Audio. Wait for the transcript to appear.

4. Read the transcript before continuing. The screen has no text editor; record again if the words are wrong.

5. Select Analyze with AI. Read the summary and any similar cases returned by the service.

The report has sections for symptoms, duration, severity, possible causes, warnings, a plain explanation, a doctor summary and next steps. Empty sections can mean the information was missing or the analysis service could not produce it.

Similar Cases shows text matches from Pinecone, a separate case search service. Its score measures similarity between text entries. It is not a diagnosis or a medical risk percentage. The current entry can also appear because the app saves it before searching.

Clear Session removes the current voice transcript and results from this browser session. It does not delete database history or Pinecone case entries.

Read saved history

Open History to see Uploaded Image Reports and Voice Assistant Medical Reports. Records are listed newest first. The image list shows the filename, date and report. The voice list shows the transcript and analysis.

History needs a working PostgreSQL database. Voice analysis also writes database records before it returns a successful result, so a database error can stop that workflow.

Clear History asks for confirmation, then deletes all records in the image and voice report tables. The delete operation is shared across the database, not limited to the current browser. It does not clear separate interaction logs, Pinecone cases or browser session data.

Understand where information goes

The backend sends images, audio or text to OpenAI services for processing. Image and voice report records can be stored in PostgreSQL. Voice analysis can also be stored in Pinecone when that connection is configured.

Some screen results are kept in browser session storage, while the theme is saved locally. The app has no built-in sign-in or patient access controls. Use approved test data while access and storage settings are being prepared.

Review risk and charts

Run a risk prediction

1. Open Risk Predictor and enter a Visit Count, such as 4.

2. Use Search Patient ID or Name to narrow the matching list. Only patients with exactly that visit count are listed.

3. Select a patient from the list. Typing a name alone does not select a patient.

4. Select Predict Risk. Review Patient Details, Prediction Result and Doctor Summary.

The patient list comes from the bundled backend CSV. For prediction, the backend tries a database record first, then the CSV. Uploading a scan does not automatically add a patient to this list or run a risk prediction.

Understand the result

Field	Meaning
Risk Class	The class number returned by the saved XGBoost model. The screen displays the number without a clinical label.
Probability	The model probability for class 1, shown as a percentage. It is not automatically the confidence in whichever class is displayed.
Accuracy	A saved overall model metric. The supplied value is 70%. It is not measured again for the selected patient.
Doctor Summary	A text explanation based on the patient record, prediction and available factor estimates.
Recommendation	Text selected from the patient record field target_urgent_followup. It is separate from the model prediction.

The prediction uses the saved patient record, including age, BMI, health indicators, smoking status and visit count. Missing fields may receive default values. Check Patient Details before interpreting the result. The screen does not let you edit these values.

Read graphical insights

Graphical Insights shows visit counts, risk class counts, heart disease, smoking status and diabetes. Hover over a chart to view values.

These charts read the bundled frontend file healthvisionpro_dataset_1000.csv, which contains 1,000 rows. They do not update from uploaded scans, voice sessions or saved history. The risk chart counts the existing target_urgent_followup labels, rather than new predictions.

The chart data and Risk Predictor data are separate copies of the CSV. If the dataset changes, a developer needs to keep both copies in sync.

Set up and support the app

Prepare the services

Local setup needs Python 3.11, Node.js 20.19 or newer in the 20 series, or Node.js 22.12 or newer, plus access to OpenAI. PostgreSQL is needed for saved history and the complete voice workflow. Pinecone is used for similar cases.

In backend/.env, set OPENAI_API_KEY and DATABASE_URL. Set PINECONE_API_KEY if using case search. Keep these values private. FRONTEND_ORIGINS controls the allowed frontend address; the local default is http://localhost:5173.

The database needs tables matching the code, including imaging_results, voice_results and interactions. The archive has no database creation script. The risk lookup also has a separate fixed local database connection that a developer must update or replace for the target environment.

Start locally on Windows

Extract the ZIP. From the HealthVisionPro-main folder, run these commands in a terminal after configuring the services above:

```
cd backend
py -3.11 -m venv .venv
.venv\Scripts\python -m pip install -r ..\requirements.txt
.venv\Scripts\python -m pip install uvicorn python-dotenv python-multipart
.venv\Scripts\python -m uvicorn main:app --reload --port 8000
```

In frontend/.env.local, set VITE_API_BASE=http://localhost:8000. Then open a second terminal in HealthVisionPro-main/frontend:

```
npm ci
npm run dev
```

Open http://localhost:5173. Check http://localhost:8000/health for {"status":"ok"}. API help is at http://localhost:8000/docs. A healthy API response confirms the server is up; test each connected service separately.

Fix common problems

Problem	What to check
Report unavailable	Check the OpenAI key, model access and backend logs.
No voice transcript	Allow the microphone, record again and check the transcription service.
History or voice fails	Check DATABASE_URL, table structure and database access.
No patients listed	Try a visit count from 1 to 10 and check the backend dataset file.
Works locally only	Multiple-image uploads and Clear History use a fixed localhost:8000 address. Update these calls before remote hosting.